Dictionary Comprehensions

{key: value for item in iterable if condition} – build or reshape dictionaries in one line.

Basics

```
names = ["ann", "bob", "cy"]
print({n: len(n) for n in names})
print({n: n.title() for n in names if n != "bob"})
print({i: n for i, n in enumerate(names)})
```

```
OUTPUT
{'ann': 3, 'bob': 3, 'cy': 2}
{'ann': 'Ann', 'cy': 'Cy'}
{0: 'ann', 1: 'bob', 2: 'cy'}
```

From two lists, then reshape with items()

```
keys, vals = ["a", "b", "c"], [1, 2, 3]
d = {k: v for k, v in zip(keys, vals)}
print(d)
# change values
print({k: v * 10 for k, v in d.items()})
# filter
print({k: v for k, v in d.items() if v > 1})
print({v: k for k, v in d.items()})          # invert
```

```
OUTPUT
{'a': 1, 'b': 2, 'c': 3}
{'a': 10, 'b': 20, 'c': 30}
{'b': 2, 'c': 3}
{1: 'a', 2: 'b', 3: 'c'}
```

Conditional values, transformed keys

```
scores = {"ann": 82, "bob": 47, "cy": 65}
print({n: ("pass" if s >= 50 else "fail")
       for n, s in scores.items()})
print({n.upper(): s for n, s in scores.items()})
```

```
OUTPUT
{'ann': 'pass', 'bob': 'fail', 'cy': 'pass'}
{'ANN': 82, 'BOB': 47, 'CY': 65}
```

Counting and grouping

```
text = "banana"
counts = {ch: text.count(ch) for ch in text}
print(counts)

words = ["ox", "cat", "dog", "bee", "emu"]
by_len = {n: [w for w in words if len(w) == n]
          for n in {len(w) for w in words}}
print(by_len)
```

```
OUTPUT
{'b': 1, 'a': 3, 'n': 2}
{2: ['ox'], 3: ['cat', 'dog', 'bee', 'emu']}
```

Parsing text into a dict

```
line = "host=db;port=5432;debug=1"
pairs = (p.split("=") for p in line.split(";"))
print({k: v for k, v in pairs})
# initialise counters
print({c: 0 for c in "abc"})
print(dict.fromkeys("abc", 0))       # same, shorter
```

```
OUTPUT
{'host': 'db', 'port': '5432', 'debug': '1'}
{'a': 0, 'b': 0, 'c': 0}
{'a': 0, 'b': 0, 'c': 0}
```

Cleaning and converting data

```
d = {"a": 1, "b": None, "c": 3, "d": None}
print({k: v for k, v in d.items() if v is not None})

menu = {"tea": 40, "cake": 120, "pie": 90}
print({k: round(v * 1.18) for k, v in menu.items()})
```

```
OUTPUT
{'a': 1, 'c': 3}
{'tea': 47, 'cake': 142, 'pie': 106}
```

Merging and nesting

```
a, b = {"x": 1, "y": 2}, {"y": 20, "z": 30}
# later wins (same as a | b)
print({**a, **b})
nested = {r: {c: r * c for c in (1, 2)}
          for r in (1, 2)}
print(nested)
```

```
OUTPUT
{'x': 1, 'y': 20, 'z': 30}
{1: {1: 1, 2: 2}, 2: {1: 2, 2: 4}}
```

Rules of thumb

- Keys must be **hashable** (str, int, tuple – not list).
- Duplicate keys: the **last** value wins; the first position is kept.
- The whole dict is built at once – it is not lazy.
- No transformation needed? `dict(zip(keys, vals))` is shorter than a comprehension.

Task	Pattern
invert a dict	`{v: k for k, v in d.items()}`
filter by value	`{k: v for k, v in d.items() if v > 0}`
map over values	`{k: f(v) for k, v in d.items()}`
lookup table	`{x.id: x for x in objs}`

PITFALL Inverting a dict silently drops entries when values repeat – two keys cannot share one value as a key.